Post-quantum key exchange: Part 1 summary

How fast are the new quantum-safe methods, and do they work correctly? (experiments E1 and E4)

1 Which methods we compare, and why these four

Post-quantum methods are built on two kinds of hard maths problems: **lattices** and **error-correcting codes**. Each kind comes in a **structured** version (extra patterns make it fast and compact, but give attackers more to work with) and an **unstructured** version (slower and bigger, but considered safer). We took one method of each kind:

Method	Based on	Design	Status
ML-KEM	lattices	structured (fast, compact)	official US standard (FIPS 203, 2024)
HQC	codes	structured (fast, compact)	chosen by NIST in 2025 as the backup
FrodoKEM	lattices	unstructured (cautious)	ISO standard (2026), recommended in Germany
Classic McEliece	codes	unstructured (cautious)	ISO standard (2026), oldest unbroken design (1978)

This shows two things at once: lattices vs. codes, and **what the extra caution costs**. Each method is tested at three strength levels (NIST levels 1, 3 and 5, as strong as AES-128, -192 and -256), giving 12 versions. They are compared with **X25519** (today's method) and the **hybrid X25519MLKEM768** (both together, used by browsers today).

2 What is a KEM?

All four methods are **KEMs** (key-encapsulation mechanisms). A KEM agrees on a shared secret in three steps:

1. **KeyGen**: the client makes a key pair and sends the public key.
2. **Encaps**: the server makes a random secret, locks it with the public key and sends it back.
3. **Decaps**: the client unlocks it with its private key. Both sides now have the same secret.

X25519 comes in two forms, and we timed both:

- **X25519 DH** (plain Diffie–Hellman, used in TLS today): each side combines its own private key with the other's public key, and the result is the shared secret. It is not a KEM, but it has the same three steps: the client makes a key pair (KeyGen); the server makes its own key pair and computes the secret (Encaps); the client computes the same secret (Decaps).
- **X25519 KEM** (DHKEM, RFC 9180, used in HPKE): the same DH calculation inside the KEM interface, plus one extra step that hashes the result together with both public keys. That gives a cleaner secret, but costs 10–14% more time. Same 64 bytes sent.

All comparisons use X25519 DH: it is what TLS uses and it is the faster one, so it is the stricter baseline.

The hybrid (X25519MLKEM768, used by browsers today) is packaged as **one KEM**, but inside it is **plain X25519 DH plus ML-KEM-768**, not DHKEM. The client sends an ML-KEM public key and an X25519 public key (1,216 bytes); the server answers with an ML-KEM ciphertext and its own X25519 public key (1,120 bytes). The two secrets are simply joined, and TLS hashes them later. It stays safe if either part is broken.

3 How we measured

- **Two computers, same software:** an **Apple M4** MacBook Air (ARM) and an **Intel i7-1185G7** laptop (x86). Both run Linux, the M4 in Docker and the Intel in WSL2, with the same compiler and libraries (liboqs 0.16.0, OpenSSL 3.5.8). Only the processor differs.
- **Three versions of the code: portable** (plain C that runs anywhere), **optimised** (uses special CPU instructions that process many numbers at once: AVX2 on Intel, NEON on ARM) and **OpenSSL** (the library real websites use, for X25519, ML-KEM and the hybrid). Optimised code exists for ML-KEM on both computers, and for FrodoKEM and McEliece only on Intel. HQC has none.
- **Timing:** every step is run 1,000 times and each run is timed. We report the **median** (the middle value, so a rare slow run does not distort it) and the **spread** of the middle half of the runs.
- **Fair conditions:** a 200 ms warm-up before timing (the M4 needs about 90 ms to reach full speed), a 30-second

pause between methods (the fanless Air must cool down), and a measurement of the processor's actual speed. Results can then also be given in **CPU cycles**, the unit used in published papers.

- **McEliece key generation is timed 100 times, not 1,000.** One run takes 40–500 ms, so 1,000 runs would add hours. Its time also varies a lot: it picks a random secret and tries to build the public key, which works only about 29% of the time. Otherwise it starts again. The number of retries is luck, so one run can take six times as long as the next. 20 samples gave an unstable median; 100 give a stable one.

4 E4: is the code correct?

Yes: all 69 checks pass on both computers. The fast optimised code gives exactly the same results as the plain code, so the speed comparison is fair.

What is checked	Applies to	M4: passed	Intel: passed
1. Answers match known correct results	all 12 KEMs, both versions	24/24 (24 cases)	24/24 (24 cases)
2. NIST official FIPS 203 test vectors	ML-KEM only	6/6 (360 cases)	6/6 (360 cases)
3. Random exchanges give the same secret	all 12 KEMs, both versions	24/24 (918 cases)	24/24 (918 cases)
4. Keys from one version work with the other	all 12 KEMs	12/12 (24 cases)	12/12 (24 cases)
5. liboqs and OpenSSL work together	ML-KEM only	3/3 (6 cases)	3/3 (6 cases)
Total		**69/69 (1,332 cases)**	**69/69 (1,332 cases)**

Table 1. Correctness checks (E4). "All 12 KEMs" = 4 methods × 3 strength levels; "both versions" = portable and optimised code. **Each row is explained in Appendix A.** Not covered: NIST's input-validation tests, which liboqs does not expose.

5 E1: results

Which numbers? liboqs has two versions of each method: *portable C*, which runs on any processor, and *optimised* code that uses the processor's vector instructions (AVX2 on Intel, NEON on the M4). Both were timed on both computers and both are shown. A dash (–) means liboqs has no optimised code for that method on that processor. Comparisons with X25519 always use the **faster** of the two versions. X25519 and the hybrid are timed in OpenSSL, which is already optimised.

One whole key exchange (KeyGen + Encaps + Decaps). "vs X25519" is the time compared with X25519 DH on the same computer: below 1.0× means faster than X25519 DH.

				Apple M4 (NEON code)			**Intel i7 (AVX2 code)**		
Method	Level	Bytes sent	Memory	Portable	Optimised	vs X25519	Portable	Optimised	vs X25519
X25519 DH (today)	–	64	–	–	62.5 µs	1.00×	–	115 µs	1.00×
X25519 KEM (DHKEM)	–	64	–	–	71.5 µs	1.14×	–	126 µs	1.10×
Hybrid X25519+ML-KEM-768	–	2,336	–	–	100 µs	1.60×	–	191 µs	1.66×
ML-KEM-512	1	1,568	10	26.8 µs	17.6 µs (1.53×)	0.28×	64.3 µs	20.8 µs (3.1×)	0.18×
ML-KEM-768	3	2,272	15	42.7 µs	28.0 µs (1.53×)	0.45×	98.6 µs	33.2 µs (3.0×)	0.29×
ML-KEM-1024	5	3,136	21	64.5 µs	42.4 µs (1.52×)	0.68×	143 µs	46.4 µs (3.1×)	0.40×
HQC-1	1	6,674	62	3.39 ms	–	54×	7.41 ms	–	65×
HQC-3	3	13,492	127	10.1 ms	–	162×	22.3 ms	–	195×
HQC-5	5	21,658	204	22.3 ms	–	357×	52.8 ms	–	461×
Frodo-640	1	19,368	75	2.70 ms	–	43×	6.33 ms	2.23 ms (2.8×)	19×
Frodo-976	3	31,424	116	6.10 ms	–	98×	13.9 ms	4.61 ms (3.0×)	40×
Frodo-1344	5	43,216	160	11.0 ms	–	176×	25.3 ms	7.58 ms (3.3×)	66×
McEliece-348864	1	261,216	412	65.5 ms	–	1,047×	90.1 ms	38.7 ms (2.3×)	337×
McEliece-460896	3	524,316	864	159 ms	–	2,536×	276 ms	113 ms (2.5×)	983×
McEliece-6688128	5	1,045,200	1,529	461 ms	–	7,375×	592 ms	134 ms (4.4×)	1,166×

Table 2. Bytes sent, memory and time for one key exchange. Level = NIST strength level (1, 3, 5 ≈ AES-128, -192, -256). Memory = peak working memory (stack, KiB) of the most demanding step. Portable = portable C. ($n\times$) after an optimised time = how many times faster it is than portable C. – = no optimised code. "vs X25519" uses the faster version.

		Apple M4			Intel i7		
Method	Code	KeyGen	Encaps	Decaps	KeyGen	Encaps	Decaps
X25519 DH (today)	OpenSSL	13.2 µs	31.2 µs	18.0 µs	27.4 µs	57.4 µs	29.9 µs
X25519 KEM (DHKEM)	OpenSSL	15.0 µs	38.0 µs	18.5 µs	26.5 µs	58.2 µs	41.2 µs
Hybrid X25519+ML-KEM-768	OpenSSL	30.1 µs	38.9 µs	31.0 µs	59.1 µs	73.7 µs	57.7 µs
ML-KEM-512	portable	8.54 µs	8.33 µs	9.96 µs	18.5 µs	20.7 µs	25.1 µs
	optimised	5.21 µs	5.67 µs	6.71 µs	5.85 µs	6.52 µs	8.46 µs
ML-KEM-768	portable	14.5 µs	13.0 µs	15.2 µs	30.6 µs	30.9 µs	37.1 µs
	optimised	8.50 µs	8.96 µs	10.5 µs	9.60 µs	10.3 µs	13.2 µs
ML-KEM-1024	portable	22.4 µs	19.7 µs	22.5 µs	46.1 µs	44.7 µs	52.6 µs
	optimised	12.8 µs	13.7 µs	15.9 µs	13.3 µs	14.6 µs	18.6 µs
HQC-1	portable	561 µs	1.12 ms	1.71 ms	1.22 ms	2.45 ms	3.73 ms
	optimised	–	–	–	–	–	–
HQC-3	portable	1.69 ms	3.36 ms	5.09 ms	3.72 ms	7.41 ms	11.2 ms
	optimised	–	–	–	–	–	–
HQC-5	portable	3.68 ms	7.44 ms	11.2 ms	8.79 ms	17.5 ms	26.5 ms
	optimised	–	–	–	–	–	–
Frodo-640	portable	862 µs	923 µs	917 µs	2.01 ms	2.16 ms	2.15 ms
	optimised	–	–	–	681 µs	785 µs	767 µs
Frodo-976	portable	1.93 ms	2.09 ms	2.07 ms	4.45 ms	4.77 ms	4.73 ms
	optimised	–	–	–	1.44 ms	1.60 ms	1.56 ms
Frodo-1344	portable	3.49 ms	3.81 ms	3.72 ms	8.06 ms	8.65 ms	8.59 ms
	optimised	–	–	–	2.58 ms	2.59 ms	2.41 ms
McEliece-348864	portable	51.8 ms	32.5 µs	13.6 ms	71.4 ms	46.8 µs	18.6 ms
	optimised	–	–	–	38.6 ms	13.0 µs	36.5 µs
McEliece-460896	portable	134 ms	66.2 µs	24.6 ms	234 ms	90.5 µs	41.9 ms
	optimised	–	–	–	113 ms	29.3 µs	73.1 µs
McEliece-6688128	portable	413 ms	121 µs	47.6 ms	496 ms	153 µs	95.6 ms
	optimised	–	–	–	134 ms	39.4 µs	68.0 µs

Table 3. Each step separately: median time per call, for both code versions. – = no optimised code. For McEliece, KeyGen is the slow step, and the optimised Intel code makes Decaps 510–1,400× faster; for ML-KEM and HQC, Decaps is slowest. McEliece KeyGen times vary a lot from run to run (random retries, see section 3).

Limits: both computers run Linux inside a virtual machine; one processor of each kind; the Intel time conversion varied 2.4% between sessions (CPU-cycle results are not affected). **Next (Part 2):** real TLS connections under network delay and loss (E2), and tests for secret-dependent timing leaks (E3). All details and tables are in the full Part 1 report; code and data at `github.com/hsh1994-ux/pq-kem-study`. Prepared with help from Claude (Anthropic); the design and interpretation are the author's.

Appendix A The five correctness checks (Table 1)

"All 12 KEMs" means the 4 methods, each at 3 strength levels: ML-KEM-512/768/1024, HQC-1/3/5, FrodoKEM-640/976/1344 and Classic McEliece-348864/460896/6688128. **"Both versions"** means the check ran once with the portable code and once with the optimised code.

1. **Answers match known correct results** (known-answer tests). *What:* liboqs ships a fingerprint (hash) of the exact output each method must produce when started from a fixed, known "random" seed. We run our build from the same seed, hash its output and compare. *Shows:* our build computes exactly the official algorithm, bit for bit. *Count:* 12 KEMs $\times$ 2 versions = 24 checks, one comparison each.
2. **NIST official FIPS 203 test vectors** (NIST ACVP). *What:* NIST publishes test cases for ML-KEM: "given these inputs, the right key, ciphertext or shared secret is exactly this". We feed in every input and compare our output with NIST's answer, for KeyGen, Encaps and Decaps. *Shows:* our ML-KEM matches the standard itself, judged by the standard's author. *Count:* 60 cases per level $\times$ 3 levels $\times$ 2 versions = 6 checks, 360 cases. ML-KEM only, because the other three methods have no NIST standard yet.
3. **Random exchanges give the same secret** (round trips). *What:* with fresh random inputs, run KeyGen $\rightarrow$ Encaps $\rightarrow$ Decaps and check that both sides end with the same secret. *Shows:* the full key exchange works with real random inputs, not only with fixed test cases. *Count:* 50 exchanges per KEM and version (McEliece: 3, because its key generation is slow) = 24 checks, 918 cases.
4. **Keys from one version work with the other** (cross-build). *What:* make a key with the portable code, encapsulate with the optimised code, decapsulate with the portable code; then the same in the other direction. *Shows:* the fast and the plain versions implement exactly the same algorithm. A version that was only consistent with itself would pass check 3 but fail here. This is what makes the speed comparison fair. *Count:* 12 KEMs $\times$ 2 directions = 12 checks, 24 cases.
5. **liboqs and OpenSSL work together** (interoperability). *What:* a key made by liboqs is encapsulated by OpenSSL and decapsulated by liboqs; then the reverse. *Shows:* two independently written implementations of the ML-KEM standard agree, so a client using one library can talk to a server using the other, as on the real internet. *Count:* 3 levels $\times$ 2 directions = 3 checks, 6 cases. ML-KEM only, because OpenSSL implements only ML-KEM among the four methods.

Total: 69 checks and 1,332 cases on each computer, all passed on both. Checks 1–2 prove that the code matches the reference and the standard; check 3 that it works with real random inputs; checks 4–5 that different implementations agree with each other.